Node Stack – Complete L1 Interview Guide

This guide is an English-only, concise revision PDF for Node Stack L1 interviews. It includes HR/introduction prompts, HTML & web basics, JavaScript and TypeScript fundamentals, Node.js & Express questions, practical coding tasks, and real-world scenario questions. Answers are kept short and easy for quick revision.

Section 1: HR & Introduction (10 questions)

1. Tell me about yourself

Give a short intro: education, current role, key skills, and a recent project in 2–3 lines.

2. Why do you want this job?

Mention learning, alignment with company tech, and growth.

3. What are your strengths?

Pick 2–3 strengths related to the role (e.g., problem-solving, quick learner).

4. What are your weaknesses?

State one real weakness and how you are improving it.

5. Tell me about a project you built

Briefly say project goal, tech stack, your role, and result.

6. How do you handle team conflicts?

Listen, communicate calmly, find common ground, and involve manager if needed.

7. What is your notice period?

Answer truthfully (e.g., 1 month) and flexibility if any.

8. Are you open to learning new tech?

Yes — explain how you learn (courses, practice projects).

9. Why Node.js?

Mention JavaScript everywhere, non-blocking I/O, and fast development.

10. Any questions for us?

Ask about team, tech stack, and expectations for the role.

Section 2: HTML & Web Basics (10 questions)

11. What is HTML?

HTML is the markup language used to structure web pages.

12. What is the difference between block and inline elements?

Block elements start on a new line (div, p); inline do not (span, a).

13. What is semantic HTML?

Using meaningful tags like <header>, <nav>, <article> for better structure and accessibility.

14. What are forms and how do they send data?
Forms collect user input and send via GET or POST to a server URL.

15. What is the purpose of the <meta> tag?
To provide page metadata like charset, viewport, and SEO instructions.

16. What is crossorigin/CORS at a browser level?
Browser rule controlling cross-origin requests; server must allow via headers.

17. What is the difference between id and class?
id is unique per page; class can be used on many elements.

18. How to include CSS and JS in HTML?
Use <link> for CSS and <script> for JS (external files or inline).

19. What is the DOM?
Document Object Model: a tree representation of HTML elements accessible via JS.

20. How do you optimize a web page?
Minify assets, compress images, use caching, and reduce HTTP requests.

Section 3: JavaScript & TypeScript (15 questions)

21. What is JavaScript?
A programming language used mainly for web interactivity and backend with Node.js.

22. Difference between == and ===
== compares with type coercion; === compares without coercion (strict).

23. What is hoisting?
Variables and functions are moved to top of scope during compilation; let/const behave differently.

24. What are closures?
A closure is a function that remembers its outer scope variables even after outer function finishes.

25. What is event bubbling?
Events propagate from child to parent elements unless stopped.

26. What is the difference between var, let and const?
var: function-scoped; let/const: block-scoped; const cannot be reassigned.

27. What are arrow functions and when to avoid them?
Shorter syntax for functions; avoid when needing own 'this' or as constructors.

28. What is Promise.all?
Runs multiple promises in parallel and resolves when all succeed (or rejects if any reject).

29. What is async/await?
Syntax to write async code that waits for a Promise result; use try/catch for errors.

30. What is TypeScript?
A typed superset of JavaScript that adds static types and compiles to JS.

31. How do you define types in TypeScript?
Use annotations like: let name: string; function add(a:number,b:number):number{}.

32. What is an interface in TypeScript?
Defines the shape of an object (properties and types) for better tooling and checks.

33. What is the difference between type and interface?
Both define shapes; type is more flexible (unions), interface supports declaration merging.

34. How to make a property optional in TypeScript?
Use ?: e.g., age?: number in an interface or type.

35. What is generics in TypeScript?
Generics allow functions/classes to work with different types while retaining type safety.

Section 4: Node.js & Express (15 questions)

36. What is Node.js?
Runtime to run JS on the server using V8 engine and non-blocking I/O.

37. What is npm and package-lock.json?
npm manages packages; package-lock.json locks installed versions for reproducible installs.

38. How do you create an Express route?
app.get('/path', (req, res) => res.send('ok'));

39. What is middleware in Express?
Function that processes request/response objects and calls next() to continue flow.

40. How to parse JSON body in Express?
Use app.use(express.json()) to parse JSON request bodies.

41. What is process.env?
Object storing environment variables accessible in Node.js as process.env.VAR_NAME.

42. How to handle file uploads in Node.js?
Use middleware like multer to handle multipart/form-data and save files.

43. What is cluster module?
Helps create worker processes to use multiple CPU cores for Node.js server.

44. How to implement authentication with JWT?
Issue a signed token on login; client sends it in headers; verify token on protected routes.

45. What is streaming in Node.js?
Working with data in chunks (streams) to handle large files or network data efficiently.

46. How to connect to MongoDB?
Use official driver or Mongoose ORM: mongoose.connect(uri).

47. How to manage configuration securely?
Use env variables, secret managers, and avoid committing credentials to repo.

48. What is rate limiting and why use it?
Limit number of requests per time window to prevent abuse and DDoS.

49. How to handle uncaught exceptions?
Use process.on('uncaughtException') and logging; prefer graceful shutdown and restart.

50. How to log requests and errors?
Use middleware and logging libraries like winston or pino; send errors to monitoring tools.

Section 5: Practical / Coding Tasks (10 questions)

51. Create a basic Express GET /health endpoint
app.get('/health', (req,res)=>res.json({status:'ok'}));

52. Create POST /users to add user to in-memory array
Use app.post('/users', (req,res)=>{ users.push(req.body); res.status(201).send(); });

53. Read a JSON file and return content via API
Use fs.readFile and JSON.parse, or require() for small static JSON files.

54. Convert a callback-based function to Promise
Wrap function in new Promise((res,rej)=>{ original(args, (err,data)=> err?rej(err):res(data)) })

55. Implement pagination for GET /items
Use query params page & limit; calculate start = (page-1)*limit and slice array or use DB limit/offset.

56. Validate request body in Express
Use middleware like express-validator or Joi to check required fields and types.

57. Write a simple middleware to block requests from a specific IP
app.use((req,res,next)=>{ if(req.ip==='1.2.3.4') return res.status(403).send('Blocked'); next(); });

58. Serve static files in Express
app.use(express.static('public')) and place files in public folder.

59. Hash a password before saving
Use bcrypt: const hash = await bcrypt.hash(password, saltRounds);

60. Create a simple rate limiter middleware

Use express-rate-limit package or implement a counter per IP with expiry.

Section 6: Real-world & Scenario Questions (5 questions)

61. How would you debug a slow API?

Check logs, measure DB queries, profile code, add caching, and identify blocking calls.

62. How to handle deployment for Node apps?

Use process managers (PM2), containers (Docker), and CI/CD pipelines for automated deploys.

63. How to scale Node.js application?

Use load balancers, clustering, horizontal scaling, and stateless services with shared sessions store.

64. How to improve security of user data?

Use HTTPS, encrypt sensitive data, validate inputs, and follow least privilege for DB access.

65. How to keep learning after joining?

Follow docs, build small projects, read release notes, and join developer communities.